Homework plus: LLMs, a famous bug, and classes with state

Deadline: **November 24, 2026**. Do the homework first: this part uses the same commands and ideas. Write your answers in ANSWERS.md.

1. Invariants from an LLM

`LlmLoops.java` has four correct methods with complete contracts and no loop invariants: `count`, `reverse`, `isqrt` and `dutchFlag`. For each one:

1. Give an LLM the method with its contract and ask for the JML `loop_invariant` and `decreases` clauses.
2. Paste them and run `./esc plus/LlmLoops.java --method NAME`.
3. When OpenJML rejects them, give the LLM the error message and try again — at most three rounds, then fix them yourself.

- **Q** For each method: which LLM, how many rounds, what kinds of mistakes (syntax, false invariant, true but too weak, missing `decreases`, …), and who had the last word. (Fill in the table in ANSWERS.md.)
- **Q** Why is it safe to use invariants written by an LLM here, even if the LLM is often wrong?

2. The Zune bug

On December 31, 2008, every Microsoft Zune 30 (a music player) froze for the whole day. The cause was the loop in `Zune.yearOf`, which turns the number of days since January 1, 1980 into a year (translated here from C to Java). `ZuneCheck.yearOfInRange` checks it for every day of the first 30 years.

Run `./bmc ZuneCheck.yearOfInRange --unwind 31 --no-unwinding-assertions`.

- **Q** It says SUCCESSFUL. Does that mean `yearOf` is correct for every day it checked?

Run it again without `--no-unwinding-assertions`.

- **Q** What does JBMC report now? Look at the values of `days` in the counterexample: is K too small, or is something else wrong? (Try a larger K.)
- **Q** What is wrong with the loop? Which date froze the Zune?

Fix `yearOf`.

- **Q** With your fix, what is the smallest K that gives VERIFICATION SUCCESSFUL with unwinding assertions? What exactly has been proved?

`yearOfInRange` only checks that the year is in range. Write `ZuneCheck.yearOfIsRight`: the day falls in the year that `yearOf` returns (use `daysBefore`). Run it with the same K.

- **Q** A colleague "fixes" the bug by changing `days > 366` to `days >= 366`. Try it: which of the two harnesses notices, and why? Then undo it.

★ `yearOf` also has a JML contract and a loop invariant. Run `./esc plus/Zune.java` on the original code: why does it say OK? Add `//@ decreases days;` above the loop and run it again. (Lecture: partial and total correctness.)

3. Range

A record with a class invariant. Run `./esc plus/Range.java`.

- **Q** Why does `size()` not verify? Give concrete values.

Write the harness `RangeCheck.size` and run it with `--unwind 1`.

- **Q** Compare JBMC's counterexample with your answer.

Then fix the code (not the contract) and check it with both tools.

4. StackAr

Implement the stack of `StackAr.java` (an array and the index of the top element) and write its specification: a class invariant, and a contract for each constructor and method (`requires`, `ensures`, and `assignable` for the methods that change the stack; `\old` refers to the state before the call). Verify it with `./esc plus/StackAr.java`. (The given file already "verifies": with no contract there is nothing to prove.)

Then run `./bmc StackArCheck.operations --unwind 5 --no-unwinding-assertions`: JBMC runs any sequence of `push`, `pop` and `peek` on a stack of capacity ≤ 3 and compares every answer with a plain array and a counter.

- **Q** Introduce a typical bug in your implementation (e.g. `isFull` off by one, or `pop` that does not decrement `top`). Which tool reports it, and how? Could a bug get past OpenJML but not JBMC, or the other way around?

Undo the bug.

5. SafeLock

Read `SafeLock.java` (a hotel-room safe) and `SafeLockCheck.neverOpensAfterTooManyWrongPins`: JBMC plays the user, choosing any sequence of events; `--unwind K` bounds the number of events. Run it with `--no-unwinding-assertions` and K = 1, 2, 3, …

- **Q** What is the smallest K that finds a bug? Which sequence of events breaks the property? What is wrong in the code?

Fix `SafeLock` and run it again, still with `--no-unwinding-assertions`, with K = 8, 10 and 12.

- **Q** What are the formula size and the time for each K?

Write `SafeLockCheck.onlyAdminUnblocks` (a blocked safe leaves BLOCKED only by an `AdminReset` with the right code) and run it with `--unwind 12 --no-unwinding-assertions`.

- **Q** Does SUCCESSFUL with K = 12 mean the property holds for every sequence of events? Can unwinding assertions help here?

Before you submit

- `./esc` says OK for `LlmLoops`, `Zune` (fixed), `Range` (the code fixed, the contract unchanged) and `StackAr` (implemented, with your contracts, without the bug you introduced).
- `./bmc` gives VERIFICATION SUCCESSFUL for both `ZuneCheck` harnesses with `--unwind 30` (with unwinding assertions), for `RangeCheck.size --unwind 1`, for `StackArCheck.operations --unwind 5 --no-unwinding-assertions` and, on the fixed `SafeLock`, for both `SafeLockCheck` harnesses with `--unwind 8 --no-unwinding-assertions`.
- No `assert false;   // remove this line` is left in a harness.
- Every question in `ANSWERS.md` is answered, and the LLM table is filled in.